

Predicting the Biodegradability of Chemicals from QSAR Data

J. Bray (Reg. No. 220123839)

Abstract—This report presents a machine learning solution for predicting the biodegradability of chemicals using QSAR data. The provided dataset contains 1055 samples with 41 structural features per chemical. After applying data cleaning, normalisation, PCA, and stratified cross-validation, five classification models are trained and evaluated: Logistic Regression, SVM, Random Forest, Naive Bayes and KNN. Performance is assessed using a combination of accuracy, ROC-AUC, and computational efficiency. Additionally, a weighted ensemble model combining all five classifiers is developed to explore performance gains from model aggregation. The results indicate that while Random Forest achieves the best single-model performance (AUC = 0.930), the ensemble provides competitive results (AUC = 0.927) with improved robustness.

I. INTRODUCTION

Predicting biodegradability is essential for environmental risk assessment and supports major regulatory frameworks [1]–[3]. Quantitative structure–activity relationship (QSAR) modelling enables these predictions by using molecular structure to predict biological activity [4], [5], allowing large chemical libraries to be screened efficiently while reducing experimental resources.

The dataset used to train the models in this work is provided by Mansouri *et al.* [5], containing 1,055 chemicals described by 41 molecular features. QSAR descriptors often have strong collinearity, making dimensionality reduction useful. Principal Component Analysis (PCA) transforms correlated features into orthogonal components ranked by explained variance, improving decorrelation and reducing noise before model training [6]. This will be implemented before any model training methods.

Machine learning applications in this domain range from interpretable linear baselines to flexible non-linear ensembles [4], [7]–[9]. Five classifiers are evaluated: Logistic Regression (LR), valued for transparency and regulatory reproducibility [10]; Random Forests (RF), which handle non-linear interactions [11]; Support Vector Machines (SVM), effective for high-dimensional separation [12]; K-Nearest Neighbours (KNN) [8], [13]; and Naive Bayes (NB) [14]. A weighted ensemble combining all five classifiers is also developed to improve generalisation [15]. Stratified cross-validation maintains class balance and prevents information leakage.

Given that predictions could influence regulatory decisions and environmental policy, model interpretability and proper validation are prioritised to avoid over-reliance on automated decisions; therefore, only standard, transparent models are implemented.

II. DATA PROCESSING

A. Formatting

Initial data review revealed three key issues. First, three pairs of duplicate samples were removed. Second, features were normalised using *z-score* normalisation (Eq. 1) per fold for all models except RF to prevent bias from differently scaled features. Third, while 30.8% of data points are outliers ($z > 3$), these represent the chemical diversity and were kept in for training.

$$\mathbf{X}_{norm} = \frac{\mathbf{X} - \mu(\mathbf{X}_{train})}{\sigma(\mathbf{X}_{train})} \quad (1)$$

B. Cross-Validation

To obtain an unbiased estimate of model performance and ensure a fair comparison between classifiers, the dataset is partitioned using a stratified cross-validation strategy. Since the response variable slightly imbalanced, stratification is essential for maintaining class proportions. This prevents folds containing too few positive or negative samples.

A k -fold cross-validation where $k \in \{5, 6, 7, 8, 9, 10\}$ is used as the outer evaluation loop. This is implemented to find the best performance of hyperparameter k . In each iteration, $k - 1$ folds are combined to form a training set, and the remaining fold serves as a hold-out test set. This ensures that every sample is evaluated exactly once and yields a robust estimate of model generalisation.

This splitting strategy provides a rigorous and unbiased estimate of model performance, prevents overly optimistic results, and supports reliable comparison across the different classifiers evaluated.

C. Dimensionality Reduction Using PCA

The data has 41 features, which is not extremely high, but QSAR data is often correlated, which can affect linear and distance-based models. This is demonstrated in the left plot of Figure 2, which shows every feature with a collinearity over 0.8.

To address these issues and improve model interpretability, Principal Component Analysis (PCA) is applied via eigen decomposition of the sample covariance matrix. The data is transformed into principal-component dimensions using

$$\mathbf{Z} = \mathbf{X}\mathbf{V}_q, \quad (2)$$

where $\mathbf{V}_q$ holds the top q eigenvectors representing orthogonal directions of decreasing variance. This produces uncorrelated

features while keeping maximal variance. The proportion of variance preserved by a q -dimensional projection is:

$$\text{Variance Explained} = \frac{\sum_{i=1}^q \lambda_i}{\sum_{i=1}^p \lambda_i}. \quad (3)$$

This can be seen in a scree plot (Figure 1). Selecting enough features to reach 95% cumulative variance is a good threshold for retaining most of the meaningful chemical information therefore $q = 22$.

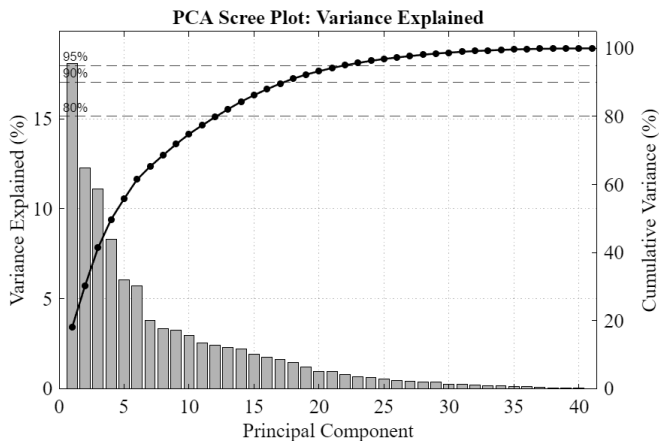


Fig. 1. Scree plot showing eigenvalues (explained variance) for each principal component. The elbow indicates an appropriate choice of retained components.

By removing redundant directions, PCA eliminates collinearity, reduces overfitting risk, and simplifies the optimisation landscape for models like LR, KNN, and SVM. The result is shown in Figure 2, with the second plot being the selected features demonstrating no collinearity over the threshold of 0.8. To ensure validity, PCA is always fitted on the training folds only, and the transformation is applied to the corresponding validation or test sets. This prevents information from the test data influencing the learned principal directions.

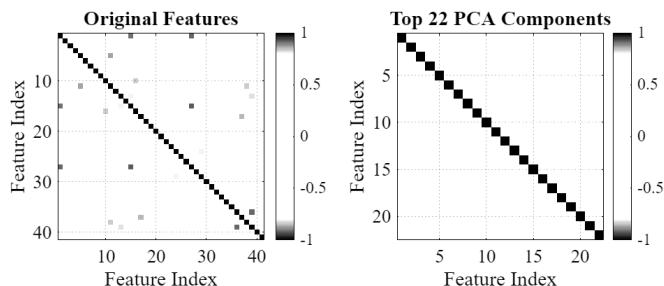


Fig. 2. A correlation matrix of the remaining features after they have been compressed into orthonormal feature dimensions via PCA.

III. METHODOLOGY

A. Models

1) *Logistic Regression*: Logistic Regression is a linear probabilistic classifier that models the probability of an outcome using the sigmoid function. It is a strong baseline

in QSAR because of its interpretability and efficiency. The probability of a positive prediction ($y = 1$) given a feature vector x is defined as:

$$P(y = 1|x) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}, \quad (4)$$

where $\mathbf{w}$ represents the weights and b is the bias. The model minimises regularised cross-entropy loss, with the regularisation parameter λ and type (Lasso L_1 or Ridge L_2) optimised via Bayesian optimisation using `fitclinear` (Section III-C).

2) *Support Vector Machine (SVM)*: Support Vector Machines (SVM) are margin-based classifiers that find a hyperplane which maximises the separation distance between the classes. Non-linear boundaries are common in chemical data, so the kernel trick is utilised to map data into higher-dimensional feature spaces. The optimisation problem is solved in its primal form:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i, \quad (5)$$

subject to $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$, where ξ_i are slack variables allowing for soft margins. The hyperparameter C (`BoxConstraint`) controls the trade-off between maximising the margin and minimising misclassification.

Linear, Polynomial, and Radial Basis Function (RBF) kernels were tested. Equation 6 demonstrates the RBF kernel equation.

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2), \quad (6)$$

where $\gamma = \frac{1}{2\sigma^2}$ is inversely related to the `KernelScale` σ . The parameters C , σ , and the kernel function type are optimised in `fitsvm` (Section III-C).

3) *Random Forest*: Random Forest is an ensemble method utilising Bootstrap Aggregating (Bagging) to reduce variance and overfitting. It constructs B decision trees and each are trained on a bootstrap sample. At each node split, a random subset of features is considered, which stops the trees from focusing on the same strong features.

The split quality is measured using the Gini impurity criterion:

$$G = 1 - \sum_{k=1}^K p_k^2, \quad (7)$$

where p_k is the proportion of samples belonging to class k at a given node. The final classification is determined by the majority vote of the ensemble:

$$\hat{y} = \text{mode}\{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_B\}. \quad (8)$$

Key hyperparameters optimised in `fitcensemble` (Section III-C) include the number of trees (`NumLearningCycles`), the minimum leaf size (`MinLeafSize`) to control tree depth, and the maximum number of splits (`MaxNumSplits`).

4) *Naive Bayes*: The Naive Bayes classifier is a probabilistic model that uses Bayes' theorem. It makes the assumption that all features are independent of each other once the class is known. It then computes the probability of each class given the input features as: The predicted class maximises this posterior probability. To model the likelihood $P(x_j|y)$, both Gaussian distributions and Kernel Density Estimation (KDE) are evaluated. Using `fitcnb`, the `DistributionNames` (Gaussian vs. Kernel), kernel type, and bandwidth (`Width`) are optimised (Section III-C).

5) *K-Nearest Neighbors (KNN)*: K-Nearest Neighbors (KNN) is a non-parametric algorithm that classifies data points based on the majority class of their k nearest training samples. Euclidean distance with inverse distance weighting is employed to prioritise closer neighbours, where the predicted class maximises the weighted vote across the k nearest neighbours $N_k(\mathbf{x})$. The number of neighbours (`NumNeighbours`), distance metric, and weighting scheme are optimised via `fitcknn` (Section III-C).

6) *Weighted Ensemble*: To capitalise on the strengths of the individual classifiers, a weighted ensemble approach is implemented that combines probabilistic outputs from all five base models. Unlike simple majority voting, this approach weights each model's contribution based on its cross-validation performance.

For a test sample $\mathbf{x}$, each base classifier M_m produces a probability estimate $\hat{p}_m(\mathbf{x})$ for the positive class. The ensemble prediction is computed as:

$$\hat{p}_{ens}(\mathbf{x}) = \sum_{m=1}^5 w_m \cdot \hat{p}_m(\mathbf{x}), \quad (9)$$

where the weights w_m are normalised to sum to 1.

The weights are computed from each model's Area Under the ROC Curve (AUC) on the full dataset:

$$w_m = \frac{\text{AUC}_m^2}{\sum_{j=1}^5 \text{AUC}_j^2}. \quad (10)$$

Squaring the AUC amplifies the contribution of better models while still keeping information from the weaker ones. This provides a balance between robustness and performance. The final class prediction uses the optimal threshold determined via Youden's J statistic on the ensemble's aggregated scores.

B. Training pipeline

To thoroughly test the selected models, a training pipeline was implemented as shown in Algorithm 0. This iterates through the five model types and then the set of cross-validation folds k to assess performance.

The core model evaluation logic is in the `EvaluateModel` function (Algorithm 0). To ensure the validity of the results and prevent data leakage, all data-dependent preprocessing steps like Z-score normalisation and PCA transformation are isolated within the cross-validation loop. These are fitted exclusively on the training data (X_{train}) and then applied after training to the hold-out test set (X_{test}). Final performance metrics are derived by combining predictions from all folds to construct a global confusion matrix and ROC curve.

Algorithm 1 Main Training Pipeline

```

1: Load and preprocess data  $X, y$ 
2: Remove duplicates
3: Visualise correlation and PCA
4: for each model  $m \in \{\text{log, svm, rnd, bay, knn}\}$  do
5:   for  $k \in [5, 10]$  do in parallel
6:      $(\text{model}_k, \text{perf}_k) \leftarrow \text{EvaluateModel}(X, y, k, m)$ 
7:   end for
8:   Select best  $k$  by maximum AUC
9:   Report performance metrics
10: end for
11: Generate ensemble predictions:  $\hat{p}_{ens} = \sum w_m \hat{p}_m$ 
12: Report ensemble performance metrics

```

Algorithm 2 EvaluateModel(X, y, k, m, q)

```

1: Create  $k$ -fold stratified partition
2: for fold  $\in [1, k]$  do in parallel
3:   Split data:  $X_{\text{train}}, X_{\text{test}}, y_{\text{train}}, y_{\text{test}}$ 
4:   Preprocess: normalise and apply PCA
5:   Train model  $m$  with hyperparameter optimisation
6:   Predict:  $\hat{y}, \text{scores} \leftarrow \text{predict}(m, X_{\text{test}})$ 
7:   store fold predictions
8: end for
9: aggregate predictions from all folds
10: find optimal threshold via Youden's  $J$  statistic
11: compute ROC, AUC, confusion matrix, and metrics
12: return trained model, performance

```

C. Hyperparameter Optimisation in MATLAB

When the `OptimizeHyperparameters` option is enabled in MATLAB for classification models, the software performs a **Bayesian optimisation** procedure by default to automatically select the best combination of hyperparameters. Each model type tunes different parameters according to the model, these are all outlined in Table III-C

TABLE I
THE PARAMETERS MATLAB OPTIMISES FOR THE CLASSIFIERS USED.

Classifier	Key Parameters
LR	Lambda: regularisation strength for penalty Regularisation: type of penalty L1 or L2
SVM	BoxConstraint: tradeoff between margin & errors KernelFunction: type of kernel mapping data KernelScale: scaling factor for kernel values
RF	NumLearningCycles: number of trees in ensemble MinLeafSize: minimum samples required per leaf MaxNumSplits: maximum allowed splits per tree
NB	DistributionNames: assumed distribution Width: kernel width for density estimation Kernel: type of kernel function used
KNN	NumNeighbors: number of neighbors considered Distance: distance metric for neighbors comparison DistanceWeight: weighting function

This optimisation is tested on an internal cross validation loop of $k = 10$, and this is iterated through 20 times to find the best parameters. The iterations can be changed but experimentally, this value is a reasonable trade-off between

processing time and model performance, as parallelisation of the for loops reduces this to a tractable time frame.

IV. MODEL ANALYSIS

A. Cross-Validation Performance

The classification accuracy trends shown in Figure 3 are similar to the AUC results in Figure 4, with RF consistently achieving the highest accuracy ($\approx 87\%$). Notably, LR is the second-best performer, often outperforming the more complex SVM and KNN models. This reinforces how effective PCA is in the analysis of these features, as it allows simple linear classifiers to capture the information more effectively without the need for complex non-linear kernel mappings.

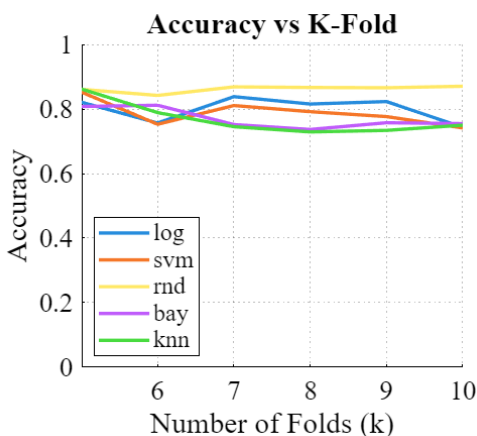


Fig. 3. Accuracy of the models over different k-fold configurations

Figure 4 shows the Area Under the Curve (AUC) performance across varying numbers of folds. The RF shows a stable performance across all k again followed by LR and SVM also showing strong stability. On the other hand, NB exhibits significant volatility and lower performance ($AUC \approx 0.85$), suggesting that the independence assumption or the specific kernel density estimations are sensitive to the partitioning of the training data.

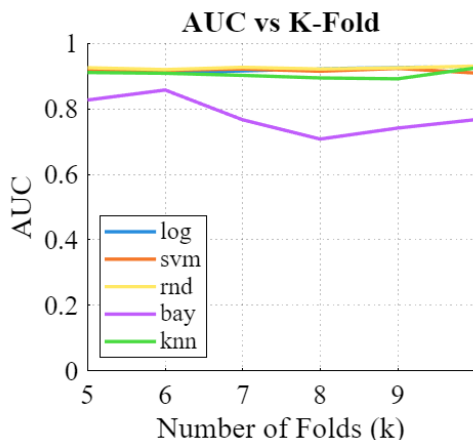


Fig. 4. AUC of the models over different k-fold configurations

Table II shows the best model performances based on AUC and the best k value found for each iteration. While LR and KNN have excellent AUC scores, their Sensitivity is extremely low (≈ 0.35). This indicates that while they are good at classifying non-biodegradable chemicals (high specificity ≈ 0.94), they don't correctly identify the majority of the true biodegradable compounds. This suggests their thresholding is very conservative, making them poor choices if identifying true positives is critical.

TABLE II
PERFORMANCE COMPARISON OF BEST MODELS BY AUC SCORE.

Model	k	AUC	Accuracy	Sensitivity	Specificity
RF	10	0.9302	0.8707	0.8446	0.8840
LR	10	0.9295	0.7433	0.3503	0.9427
KNN	10	0.9244	0.7500	0.3475	0.9542
SVM	9	0.9232	0.7766	0.4718	0.9312
NB	6	0.8573	0.8118	0.7599	0.8381
ENS	5	0.9269	0.8517	0.8644	0.8453

B. ROC Curves

The Receiver Operating Characteristic (ROC) curves for the best-performing iteration of each model are presented in Figure 5. The RF classifier achieved the highest overall performance with an AUC of 0.930, followed closely by LR (AUC = 0.929) and KNN (AUC = 0.924). The ROC curves for these top three models are nearly overlapping, indicating similar sensitivity and specificity profiles.

The steep initial rise of the LR curve is particularly significant for QSAR applications, as it implies a high True Positive Rate can be achieved with a very low False Positive Rate. While RF marginally outperforms the linear model numerically, the interpretability and simplicity of LR may make it the preferable choice for a regulatory solution. NB remains the distinct underperformer (AUC = 0.857), confirming it is less suitable for this specific high-dimensional chemical feature space.

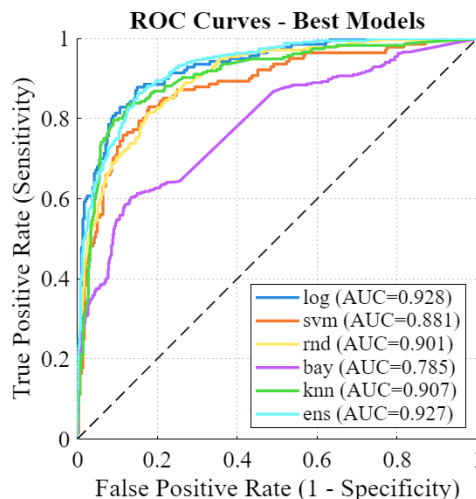


Fig. 5. ROC of the best of each model type

Table III states the confusion matrix elements for each model using the optimal threshold derived from Youden’s J statistic. The RF model has the most balanced performance, with the highest number of True Positives (299) and still a high number of True Negatives (617). This explains its high F1 score and accuracy shown in the previous analysis. The SVM model shows balanced results, with 167 True Positives and 187 False Negatives, placing it between the highly conservative LR/KNN models and the balanced RF model. Finally, NB registers the highest number of False Positives (113), which contributed to its lower AUC and specificity. The RF model’s ability to keep high True Positives while limiting False Negatives makes it superior for an application where identifying biodegradable compounds is important.

TABLE III
CONFUSION MATRIX STATISTICS FOR BEST PERFORMING MODELS.

Model	TP	TN	FP	FN
Random Forest	299	617	81	55
Logistic Regression	124	658	40	230
K-Nearest Neighbors	123	666	32	231
Support Vector Machine	167	650	48	187
Naive Bayes	269	585	113	85
Ensemble	306	590	108	48

C. Ensemble Model Performance

The ensemble achieved an AUC of 0.927, which ranks between the RF (0.930) and LR (0.929) models. While it couldn’t beat the RF, it demonstrates several important characteristics.

The confusion matrix shows the ensemble correctly identified 306 biodegradable compounds with 590 true negatives, yielding an accuracy of 85.17%. Its sensitivity (0.864) and specificity (0.845) are well-balanced, reducing the FN bias shown by LR and KNN.

The weight distribution shows that LR receives the highest weight (0.222), followed closely by KNN (0.212) and RF (0.209). This is expected given these models’ strong AUC performance. The NB and SVM models still have impactful weights (0.158 and 0.200 respectively), suggesting they contribute complementary information that helps stabilise predictions across the dataset.

Benchmarking these results against the original study by Mansouri *et al.* [5], the ensemble and RF models are strong competitors. Mansouri’s best individual model (SVM) achieved a test Error Rate (ER) of 0.14 (Accuracy $\approx$ 86%), while their ‘Consensus 1’ model achieved an ER of 0.13 (Accuracy $\approx$ 87%). The RF model matches this baseline. Furthermore, the weighted ensemble has even better sensitivity of 0.864 compared to Mansouri’s consensus sensitivity of 0.820. This means that even if the overall error rate is similar, the ensemble is more effective at correctly identifying biodegradable chemicals (True Positives), addressing the need to minimise false negatives.

While the ensemble does not achieve the highest single AUC value, it reduces reliance on any single algorithm’s assumptions or weaknesses, which could be beneficial in regulatory contexts.

V. CONCLUSION AND RECOMMENDATION

A. Discussion

This study successfully trained and evaluated five diverse classifiers for predicting chemical biodegradability. The analysis provides clear insights into which models are best and the necessary preprocessing steps for QSAR data.

The RF model was proven to be the best option, with balanced performance, achieving the highest AUC and F1 score, low variance and high stability across varying cross-validation folds. On the other hand, the analysis revealed a classification bias in models like LR and KNN, which had extremely low sensitivity and high false negatives, despite high AUC values, making them unreliable for identifying true biodegradable compounds. This highlights a trade-off with this model, as LR offers greater interpretability, but its severe bias due to the slight class imbalance makes it less suitable than the RF model. Therefore, the Random Forest model is recommended where accuracy is preferred. However, the weighted ensemble model should be chosen where robustness and balanced sensitivity/specificity are more critical, as it mitigates individual model weaknesses.

The final ranking of all six approaches based on multiple performance criteria is summarised in Table IV

TABLE IV
MODEL RANKINGS ACROSS DIFFERENT EVALUATION METRICS.

Rank	AUC	Accuracy	Sensitivity	Specificity	F1
1	RF	RF	ENS	KNN	RF
2	ENS	ENS	RF	LR	ENS
3	LR	NB	NB	SVM	NB
4	KNN	SVM	SVM	ENS	SVM
5	SVM	KNN	LR	RF	KNN
6	NB	LR	KNN	NB	LR

B. Future Work

To achieve potentially higher predictive performance and gain deeper insights into the QSAR mechanisms, future work could focus on several key areas. Firstly, feature engineering could be significantly enhanced by moving beyond correlation-based PCA to employ techniques like Recursive Feature Elimination (RFE) or domain-specific descriptor subset selection, which would further optimise the quality of the input data. Secondly, having implemented a weighted averaging ensemble, future work could explore more sophisticated meta-learning approaches such as stacking or a dynamic ensemble selection, where different model combinations are used for different regions of the feature space. Finally, while the current standard machine learning models proved effective, future research could explore deep learning architectures, including neural networks or multi-layer perceptrons, to potentially capture the complex, non-linear chemical relationships. However, using these “black-box” models requires careful consideration in the regulatory context, as high transparency and reliability are required, which can be difficult to achieve with these models.

REFERENCES

- [1] U.S. Environmental Protection Agency, "Criteria for biodegradability claims on products registered under fifra," <https://www.epa.gov/pesticide-labels/criteria-biodegradability-claims-products-registered-under-fifra>, 2025, last updated August 14, 2025.
- [2] European Commission, "Reach regulation: Regulation (ec) no. 1907/2006 on the registration, evaluation, authorisation and restriction of chemicals," https://environment.ec.europa.eu/topics/chemicals/reach-regulation_en, 2006, entered into force 2007; ongoing updates.
- [3] Organisation for Economic Co-operation and Development, "Q-sar assessment framework: Guidance for the regulatory assessment of quantitative structure–activity relationship models and predictions," 2019.
- [4] J. Li, T. Zhao, Q. Yang, S. Du, and L. Xu, "A review of quantitative structure-activity relationship: The development and current status of data sets, molecular descriptors and mathematical models," *Chemometrics and Intelligent Laboratory Systems*, vol. 256, p. 105278, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0169743924002181>
- [5] K. Mansouri, T. Ringsted, D. Ballabio, R. Todeschini, and V. Consonni, "Quantitative structure–activity relationship models for ready biodegradability of chemicals," *Journal of Chemical Information and Modeling*, vol. 53, no. 4, pp. 867–878, 2013. [Online]. Available: <https://doi.org/10.1021/ci4000213>
- [6] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. Springer, 2002. [Online]. Available: <https://doi.org/10.1007/b98835>
- [7] A. Cherkasov, E. N. Muratov, D. Fourches, A. Varnek, I. I. Baskin, M. Cronin, J. Dearden, P. Gramatica, Y. C. Martin, R. Todeschini, V. Consonni, V. E. Kuz'min, R. Cramer, R. Benigni, C. Yang, J. Rathman, L. Terfloth, J. Gasteiger, A. Richard, and A. Tropsha, "Qsar modeling: Where have you been? where are you going to?" *Journal of medicinal chemistry*, vol. 57, no. 12, pp. 4977–5010, 2014.
- [8] Z. Wu, B. Ramsundar, E. N. Feinberg, J. Gomes, C. Geniesse, A. S. Pappu, K. Leswing, and V. Pande, "Do we need different machine learning algorithms for qsar modeling? a comprehensive assessment of 16 machine learning algorithms on 14 qsar data sets," *Briefings in Bioinformatics*, vol. 22, no. 4, p. bbaa321, 2021. [Online]. Available: <https://doi.org/10.1093/bib/bbaa321>
- [9] V. Svetnik, A. Liaw, C. Tong, J. C. Culberson, R. P. Sheridan, and B. P. Feuston, "Random forest: a classification and regression tool for compound classification and qsar modeling," *Journal of Chemical Information and Computer Sciences*, vol. 43, no. 6, pp. 1947–1958, 2003. [Online]. Available: <https://doi.org/10.1021/ci034160g>
- [10] D. W. Hosmer and S. Lemeshow, *Applied Logistic Regression*, 2nd ed. Wiley, 2000. [Online]. Available: <https://doi.org/10.1002/0471722146>
- [11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. [Online]. Available: <https://doi.org/10.1023/A:1010933404324>
- [12] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995. [Online]. Available: <https://doi.org/10.1007/BF00994018>
- [13] S. Khare, P. Subramani, S. Choudhari, S. Phalle, S. Kumbhar, A. Kadam, and P. B. Choudhari, "K nearest neighbor and 3d qsar analysis of thiazolidinone derivatives as antitubercular agents," *Journal of Pharmaceutical Research*, vol. 15, no. 3, pp. 67–72, 2016.
- [14] R. Singh and colleagues, "Qsar classification model for antibacterial compounds and its use in virtual screening," *Journal of Chemical Information and Modeling*, vol. 52, no. 10, 2012.
- [15] T. G. Dietterich, "Ensemble methods in machine learning," *International workshop on multiple classifier systems*, pp. 1–15, 2000.